

Analyzing the Role of AI-Assisted Programming in Modern Software Development

Assessment 2

AI-Assisted Programming

IT – 3003

Advance Programming Techniques

Ruchitha Vithana

2023s20200

S17450

Contents

Task 1 – Conceptual Understanding	3
1.1 What is AI-Assisted Programming?	3
1.2 Types AI Coding Tools	4
1.3 Advantages of AI-Assisted Programming	5
1.4 Risks and Limitations of AI-Assisted Programming	5
Task 2 – Practical Implementation	7
2.1 Problem Description	7
2.2 Version A - Traditional Programming Approach	8
2.3 Version B – AI-Assisted Programming Approach	14
Comparison between Version A and Version B	20
UML Diagrams	21
Task 3 – Critical Analysis	22
3.1 How AI changed their coding approach?	23
3.2 Whether AI Improved or Reduced Understanding	23
3.3 Cases Where AI Should Not Be Used	24
3.4 Ethical Considerations in AI-Generated Code	24
Task 4 – Conclusion	25
Repository Link	25

Task 1 – Conceptual Understanding

1.1 What is AI-Assisted Programming?

AI-assisted programming is the application of Artificial Intelligence (AI) systems to facilitate the development, improvement, and debugging of computer programs. AI tools may be able to produce code, suggest possible solutions, detect errors and clarify concepts in programming. AI systems do not act as replacements for programmers but serve as assistants for completing tasks faster and better.

In the modern era, many developers make use of AI tools, such as ChatGPT and GitHub Copilot, in order to increase the speed of software development. These tools comprehend user requests and offer code suggestions depending on the requirements stated. Nonetheless, the developers should check the suggested code and test it for its correctness.

For this task, Claude.ai was used as an AI tool to develop a Restaurant Management System in version B.

1.2 Types AI Coding Tools

There are various coding tools that are employed in developing software using artificial intelligence.

1. Code Generating Tools

These are tools that enable developers to write code based on specific instructions. They can produce functions, class definitions, and whole applications. These include ChatGPT and GitHub Copilot.

2. Code Completion Tools

These are tools that suggest what should come next when a developer is writing the code. This speeds up coding and minimizes typing mistakes.

3. Debugging Tools

These are AI-based tools that enable programmers to locate bugs in the code and suggest fixes for these problems.

4. Refactoring Tools

They help make improvements in the organization of the code without altering what it does. Refactoring tools are important for improving the organization of code.

5. Documentation Tools

1.3 Advantages of AI-Assisted Programming

There are several benefits of AI-assisted programming for software developers.

First of all, the technology allows saving development time. The AI algorithm generates the code rapidly and performs repetitive work which makes the whole process faster.

In addition, the technology improves code quality as the system suggests proper coding techniques and finds errors before the final completion of the program.

Also, AI can be considered as an effective tool for learning. It can help students and beginner programmers with understanding different aspects of programming and its techniques.

Developer productivity is increased because of the features offered by AI such as code completion, debugging, and documentation.

1.4 Risks and Limitations of AI-Assisted Programming

There are several risks associated with using AI-assisted programming despite its benefits.

First of all, sometimes AI generates wrong or misleading code. It can look good, but actually, it can have logical mistakes or employ non-existent methods. So, developers should always check the code created by the algorithm.

Secondly, programmers can get too dependent on AI. They will never develop their own programming skills if they are constantly using AI to create code.

Code generated by AI can include security flaws and violate coding standards, so developers should examine such code before implementing it in practice.

Also, sometimes AI does not know the complete requirements of a certain project. As a result, it generates code which is partially relevant.

Finally, there are ethical issues connected with the use of AI. For instance, students should use AI to learn how to code, but never just copy AI-generated code without comprehending it. Developers should also consider copyright, privacy, and proper use of the algorithm's content.

In this assignment, AI-assisted code was rather helpful, but some aspects needed to be improved in order to meet project's requirements.

During this assignment, the AI has helped to generate pieces of the Restaurant Management System, find improvements, and fix errors. Although some corrections were required, it saved a lot of time.

In conclusion, AI-assisted programming helps to develop programs faster, improves code quality, and also allows gaining additional programming skills.

Task 2 – Practical Implementation

2.1 Problem Description

In order to complete the current task, a Restaurant Management System has been developed using Java and Object-Oriented Programming (OOP) methodology. The goal of the system is to provide a convenient and efficient tool for management of restaurant operations.

The system enables users to perform typical restaurant management tasks like menu item management, customer information management, orders management, reservations management, and billing management. Instead of maintaining the records manually, the system manages and processes all the necessary information electronically to make restaurant operations fast and accurate.

The project has been completed with the use of two different methodologies:

Version A: Traditional programming methodology, which implies that the application was developed manually without any assistance from AI.

Version B: AI-assisted programming methodology, which implies that Claude.ai was used for generation and improvement of code. The code produced by AI was reviewed, edited, and tested to be incorporated into the system.

Both versions provide the same core functionality but were developed using different methods. This allows a comparison of development time, code quality, readability, error handling, and maintainability.

Main Features of the Restaurant Management System

The system includes the following features:

- Menu management
- Customer management
- Order management
- Bill generation
- Display of restaurant information
- User-friendly console interface

The project also demonstrates several Object-Oriented Programming concepts, including:

- Classes and Objects
- Encapsulation
- Inheritance
- Polymorphism

These concepts help create a well-structured, reusable, and maintainable software application.

2.2 Version A - Traditional Programming Approach

Version A of the Restaurant Management System was created using the traditional programming approach, that is without using any AI. All classes, methods, and program logic were created using traditional coding approaches.

First, the system requirements were identified, and then all necessary classes were created. Every system feature was created one-by-one, which included managing menu items, managing customers, managing orders, managing reservations, and creating bills. After the completion of every system feature, the program was tested to check for errors.

Following OOP concepts were used while developing the system:

- Classes and Objects - Classes and objects were used to represent various entities such as customers, menu items, orders, and reservations.
- Encapsulation - Private data members and getter/setter methods were used to encapsulate data.
- Inheritance - Common functionalities were inherited from parent classes to avoid code duplication.
- Polymorphism - Wherever needed, methods were overridden to change their behavior.

Manual creation of the system took more time since every feature had to be created without any automated help. Even though the traditional approach is time-consuming, it improves programming skills.

Screenshots

Screenshot of project structure

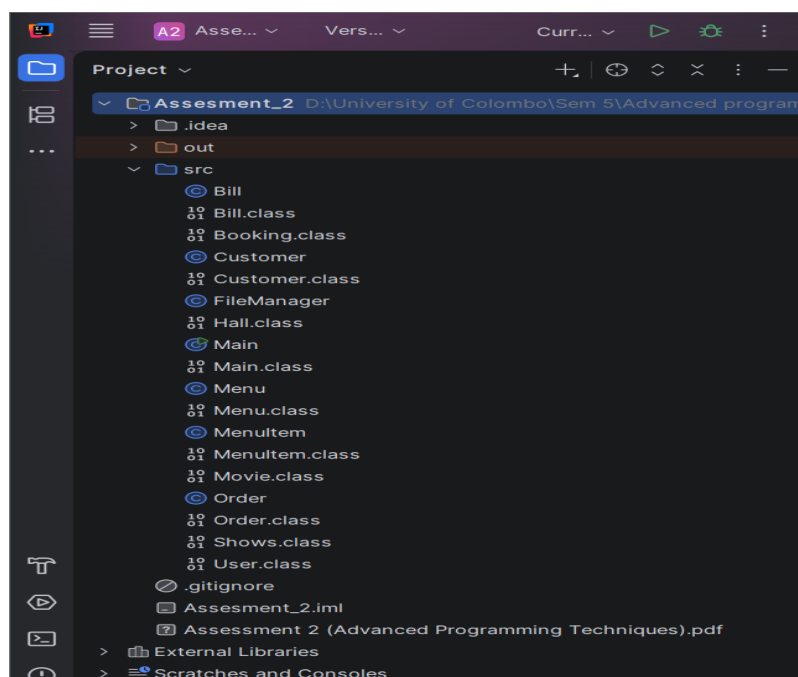


Figure 2.1

Figure 2.2 shows some of the screenshots in the Main class, which controls the execution of the Restaurant Management System and provides the menu for user interaction.

```

1  import java.util.Scanner;
2  import java.util.ArrayList;
3
4  public class Main {
5
6      public static void main(String[] args) {
7
8          Scanner input = new Scanner(System.in);
9
10         Menu menu = new Menu();
11
12         |
13         menu.addItems(new MenuItem(1001, "Chicken Burger", "Burger", 1200, true));
14         menu.addItems(new MenuItem(1002, "Cheese Pizza", "Pizza", 1800, true));
15         menu.addItems(new MenuItem(1003, "Coca Cola", "Drink", 150, true));
16
17         Customer customer = null;
18
19         ArrayList<Order> customerOrders = new ArrayList<>();
20         boolean running = true;
21         int orderCounter = 101;
22         int billCounter = 101;
23
24         while (running) {
25
26             System.out.println("\n----- Restaurant System -----");
27             System.out.println("1. Staff");
28             System.out.println("2. Customer");
29             System.out.println("3. Exit");
30             System.out.print("Enter Choice: ");
31
32             int choice = input.nextInt();
33

```

```

34         switch (choice) {
35
36             case 1:
37
38                 System.out.println("\n--- Staff Working ---");
39                 System.out.println("1. Add Item");
40                 System.out.println("2. Display Menu");
41                 System.out.println("3. Search Item");
42                 System.out.println("4. Update Price");
43                 System.out.println("5. Update Availability");
44                 System.out.println("6. Remove Item");
45                 System.out.print("Enter Choice: ");
46
47                 int staffChoice = input.nextInt();
48                 input.nextLine();
49
50                 switch (staffChoice) {
51
52                     case 1:
53
54                         System.out.print("Enter ID: ");
55                         String id = input.nextLine();
56
57                         System.out.print("Enter Name: ");
58                         String name = input.nextLine();
59

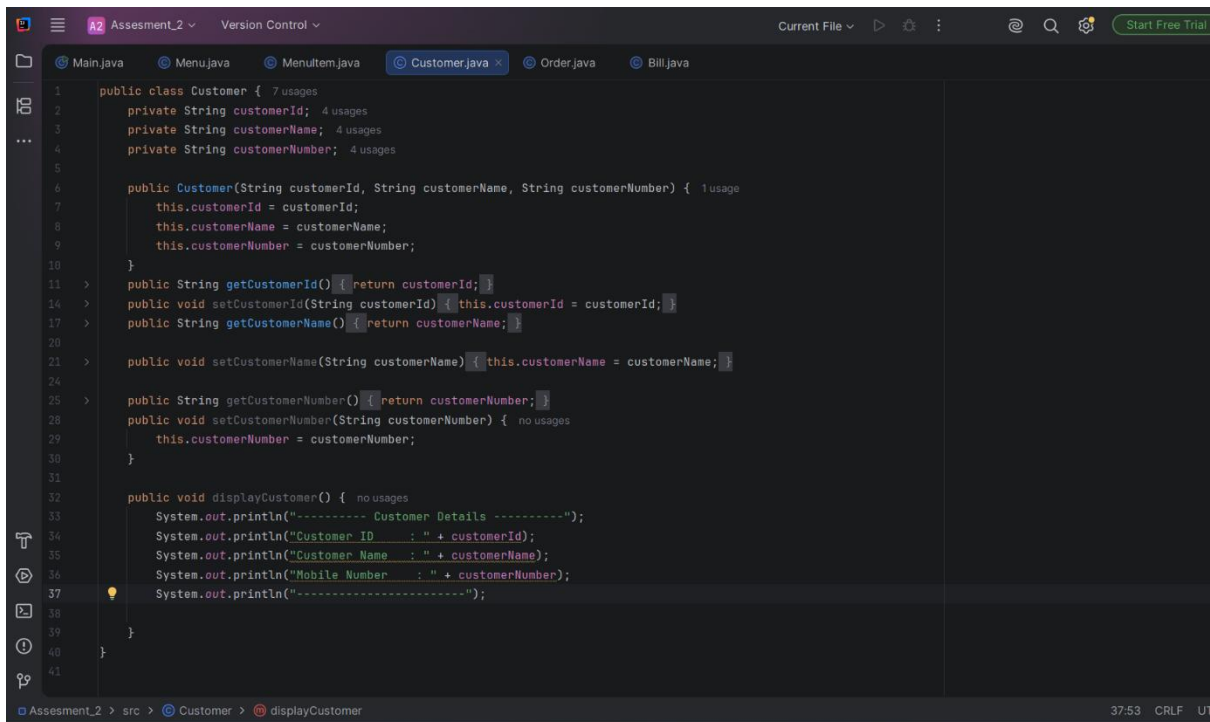
```

```

91         public static void main(String[] args) {
92             break;
93
94             case 3:
95
96                 System.out.print("Enter Item ID: ");
97                 String searchId = input.nextLine();
98
99                 MenuItem found = menu.searchItem(searchId);
100
101                 if (found != null) {
102                     found.displayItem();
103                 } else {
104                     System.out.println("Item not Found.");
105                 }
106
107                 break;
108
109             case 4:
110
111                 System.out.print("Enter Item ID: ");
112                 String updateId = input.nextLine();
113
114                 System.out.print("New Price: ");
115                 double newPrice = input.nextDouble();
116                 input.nextLine();
117
118                 menu.updatePrice(updateId, newPrice);
119
120                 break;
121

```

Figure 2.3 shows the Customer class used to store customer information. The class demonstrates encapsulation by using private attributes and public methods.



```
1 public class Customer { 7 usages
2     private String customerId; 4 usages
3     private String customerName; 4 usages
4     private String customerNumber; 4 usages
5
6     public Customer(String customerId, String customerName, String customerNumber) { 1 usage
7         this.customerId = customerId;
8         this.customerName = customerName;
9         this.customerNumber = customerNumber;
10    }
11
12    > public String getCustomerId() { return customerId; }
14    > public void setCustomerId(String customerId) { this.customerId = customerId; }
17    > public String getCustomerName() { return customerName; }
20
21    > public void setCustomerName(String customerName) { this.customerName = customerName; }
24
25    > public String getCustomerNumber() { return customerNumber; }
28    public void setCustomerNumber(String customerNumber) { no usages
29        this.customerNumber = customerNumber;
30    }
31
32    public void displayCustomer() { no usages
33        System.out.println("----- Customer Details -----");
34        System.out.println("Customer ID : " + customerId);
35        System.out.println("Customer Name : " + customerName);
36        System.out.println("Mobile Number : " + customerNumber);
37        System.out.println("-----");
38    }
39
40 }
41
```

Figure 2.3

Figure 2.4 shows the Order class, which stores details of a customer's order, including the associated Customer and MenuItem objects, along with the quantity and status. It calculates the order's total price through calculateTotalPrice () and displays the order details via displayOrder (), demonstrating both encapsulation and association between classes.

figure 2.4

```

1 public class Order { 8 usages
2     private String orderId; 4 usages
3     private Customer customer; 5 usages
4     private String orderStatus; 4 usages
5     private int orderQuantity; 5 usages
6     private MenuItem menuItem; 5 usages
7
8     public Order(String orderId, Customer customer, String orderStatus, int orderQuantity, MenuItem menuItem) { 1 usage
9         this.orderId = orderId;
10        this.customer = customer;
11        this.orderStatus = orderStatus;
12        this.orderQuantity = orderQuantity;
13        this.menuItem = menuItem;
14    }
15
16    public String getOrderId() { return orderId; }
17    public void setOrderId(String orderId) { this.orderId = orderId; }
18
19    public Customer getCustomer() { return customer; }
20    public void setCustomerId(Customer customer) { this.customer = customer; }
21    public String getOrderStatus() { return orderStatus; }
22    public void setOrderStatus(String orderStatus) { this.orderStatus = orderStatus; }
23
24    public int getOrderQuantity() { return orderQuantity; }
25    public void setOrderQuantity(int orderQuantity) { this.orderQuantity = orderQuantity; }
26
27    public MenuItem getMenuItem() { return menuItem; }
28    public void setMenuItem(MenuItem menuItem) { this.menuItem = menuItem; }
29
30 }
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73

```

Figure 2.5 shows the Bill class, which calculates a customer's final invoice by reusing the total price from the associated Order object rather than recalculating it. It then applies the tax rate and discount rate through the tax(), discount(), and grandTotal() methods to produce the final amount, demonstrating both composition and encapsulation.

Figure 2.5

```

1 public class Bill { 4 usages
2     private String billId; 4 usages
3     private Order order; 8 usages
4     private double taxRate; 4 usages
5     private double discountRate; 4 usages
6     private Customer customer; no usages
7     private MenuItem menuItem; no usages
8
9     public Bill(String billId, Order order, double taxRate, double discountRate) { 2 usages
10        this.billId = billId;
11        this.taxRate = taxRate;
12        this.discountRate = discountRate;
13        this.order = order;
14    }
15
16    public String getBillId() { return billId; }
17    public void setBillId(String billId) { this.billId = billId; }
18
19    public Order getOrder() { return order; }
20    public void setOrder(Order order) { this.order = order; }
21    public double getTaxRate() { return taxRate; }
22    public void setTaxRate(double taxRate) { this.taxRate = taxRate; }
23
24    public double getDiscountRate() { return discountRate; }
25    public void setDiscountRate(double discountRate) { this.discountRate = discountRate; }
26
27    public double calculateSubTotal() { return order.calculateTotalPrice(); }
28
29    public double tax() { 2 usages
30        return calculateSubTotal()*taxRate;
31    }
32
33    public double discount() { return calculateSubTotal()*discountRate; }
34
35    public double grandTotal() { return calculateSubTotal()+tax()+discount(); }
36
37    public double displayBill() { 2 usages
38        System.out.println("----- BILL -----");
39        System.out.println("Bill ID : " + billId);
40        System.out.println("Customer : " + order.getCustomer().getCustomerName());
41        System.out.println("Item : " + order.getMenuItem().getItemName());
42        System.out.println("Quantity : " + order.getOrderQuantity());
43        System.out.println("Subtotal : " + order.calculateTotalPrice());
44        System.out.println("Tax : " + tax());
45        System.out.println("Discount : " + discount());
46        System.out.println("Grand Total : " + grandTotal());
47        System.out.println("-----");
48
49        return 0;
50    }
51
52 }
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74

```

Figure2.6: Main Menu Output

Figure 2.6 shows the main menu of the Restaurant Ordering System, where the user selects between Staff and Customer options. This is the entry point of the program, driven by the loop in Main.java.

```
Terminal Local x + v
----- Restaurant System -----
1. Staff
2. Customer
3. Exit
Enter Choice: 1

---- Staff Catalog ----
1. Add Item
2. Display Menu
3. Search Item
4. Update Price
-----
ssesment_2 > src > Order
```

Figure 2.7: Bill Generation Output

Figure 2.8 shows the final bill produced by the displayBill() method in Bill.java, including the subtotal, tax, discount, and grand total. This confirms that Bill correctly builds on the Order object's data to generate the invoice.

```
Terminal Local x + v

Do you want to order another item? (y/n): n
----- BILL -----
Bill ID: BL101
Customer: Ruchitha
Item: Coca Cola
Quantity: 3
Subtotal: 1050.0
Tax : 105.0
Discount : 52.5
Grand Total : 1207.5
-----
```

```
Terminal Local x + v

----- BILL -----
Bill ID: BL102
Customer: Ruchitha
Item: Chicken Burger
Quantity: 2
Subtotal: 2400.0
Tax : 240.0
Discount : 120.0
Grand Total : 2760.0
-----

---- Total Bill: Rs. 3967.5 ----
```

2.3 Version B – AI-Assisted Programming Approach

The second version of the Restaurant Management System was created using ChatGPT as an AI-assisted programming tool. Prompts explaining the desired functionalities, including menu management, ordering, and billing were given to the AI, and then the outputted code was examined, tested, and revised when needed for inclusion in the final system.

The first step in developing the program was to ask the AI to generate the code for a layered project structure, which was later examined one by one to see the purpose of each package and then included in the final program. The outputted code was compiled and checked to make sure that it was doing what was expected from it.

The following OOP principles were applied in this version:

- **Classes & Objects** – Used to represent entities like Customer, MenuItem, Order, and Bill.
- **Encapsulation** – Private variables with public getters are used in the model classes.
- **Inheritance** – Custom exceptions like ItemNotFoundException inherit from Java's RuntimeException.
- **Abstraction** – MenuRepository interface outlines menu operations without implementation, whereas InMemoryMenuRepository gives the implementation.
- **Composition** – Order consists of a number of OrderLine items, each referring to MenuItem and its amount.

As compared to the Version A, the current version took less effort in terms of manually coding as the AI generated the initial structure itself, but some time would have been spent to comprehend and verify the generated code.

Screenshots

Figure 2.9 shows the package structure of the AI-assisted implementation, organized into app, model, service, repository, exception, and Ui packages that separate the program's entry point, data, business logic, and error handling into distinct responsibilities, unlike Version A's single flat package.

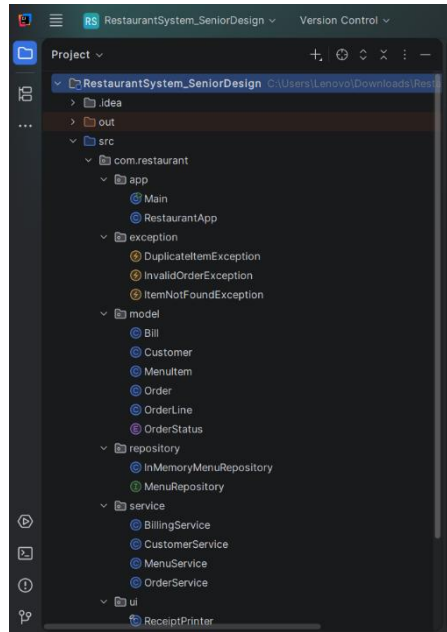


Figure 2.9

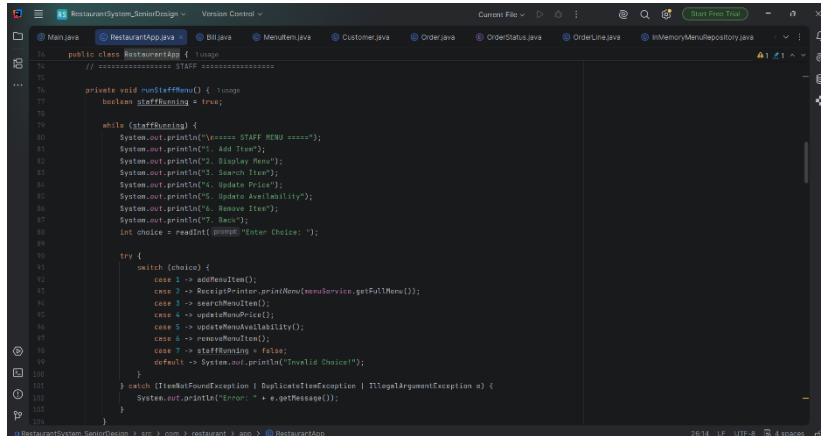
Figure 2.10: RestaurantApp.java

The RestaurantApp class, shown in Figure 2.10, functions as the entry point of the AI-assisted version and manages the flow of the entire program using a menu loop that provides two options: Staff and Customer. It does not implement any business logic within itself but rather delegates each of the actions to a specialized service class such as delegating the order processing and validation to OrderService and the bill calculation to BillingService; ReceiptPrinter takes care of user inputs and output display only. This is an important structural difference with Version A, where Main.java implemented everything in one single class.

```

1  package com.restaurant.app;
2
3  import java.util.Scanner;
4
5  /**
6   * The overall entry point. Its only job is to read input, call the
7   * right service, and send the results to the printer. It helps me
8   * enforce rules of its own, and every service call that can fail
9   * is guarded as a bad input never reaches the printer.
10  */
11
12  public class RestaurantApp {
13
14      private final Scanner input = new Scanner(System.in);
15
16      private final MenuService menuService;
17      private final CustomerService customerService;
18      private final OrderService orderService;
19      private final BillingService billingService;
20
21      private Customer currentCustomer;
22      private Order currentOrder;
23
24      public RestaurantApp() {
25          MenuRepository repository = new InMemoryMenuRepository();
26          MenuService menuService = new MenuService(repository);
27          OrderService orderService = new OrderService(menuService);
28          BillingService billingService = new BillingService(menuService, orderService);
29      }
30
31      private void readMenu() {
32          menuService.addMenu(0, "Burger", "Chicken Burger", 10.99, 1200, 1000, true);
33          menuService.addMenu(1, "Pizza", "Cheese Pizza", 8.99, 1000, 1000, true);
34          menuService.addMenu(2, "Salad", "Greek Salad", 5.99, 500, 1000, true);
35      }
36  }

```



Figures 2.10

Figure 2.11: OrderLine.java

Figure 2.11 shows the OrderLine class, which represents a single item and its quantity within an order, validating in its constructor that the menu item is not null and the quantity is greater than zero. Unlike Version A's Order class, which only supports one menu item per order, this design allows an order to hold multiple OrderLine objects, enabling customers to order several different items at once.

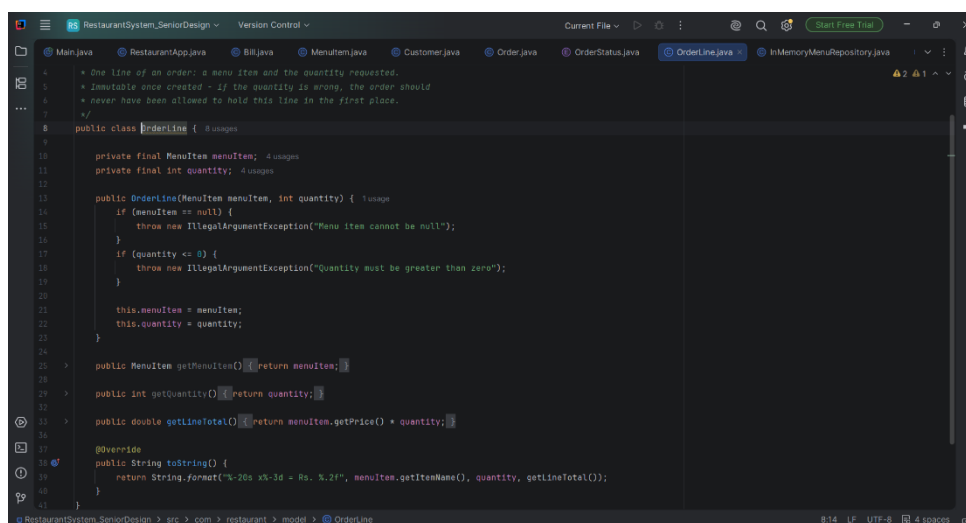
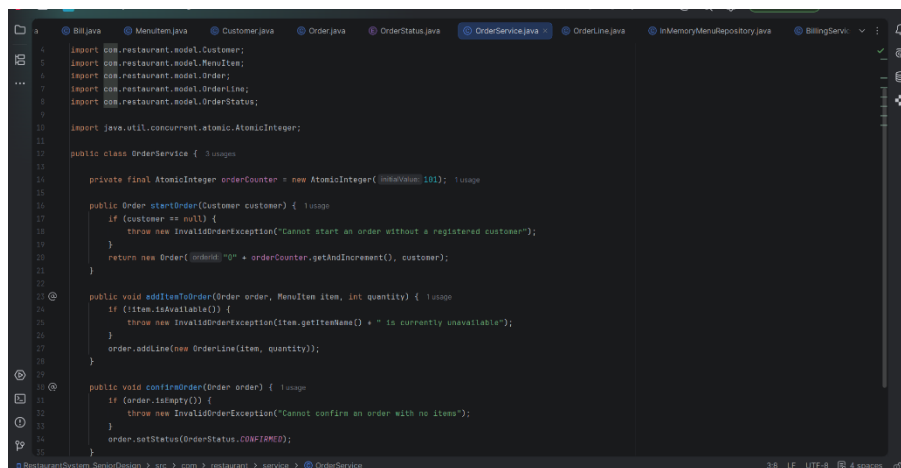


Figure 2.12: OrderService.java

Figure 2.12 shows the OrderService class, which manages order creation and validation by starting a new order for a registered customer, adding items to it, and confirming it only if the order is not empty. It throws an InvalidOrderException when an order is started without a customer or an item is unavailable, demonstrating a service-layer validation pattern that Version A does not have, since Version A's Main.java performs this logic directly without dedicated error handling.

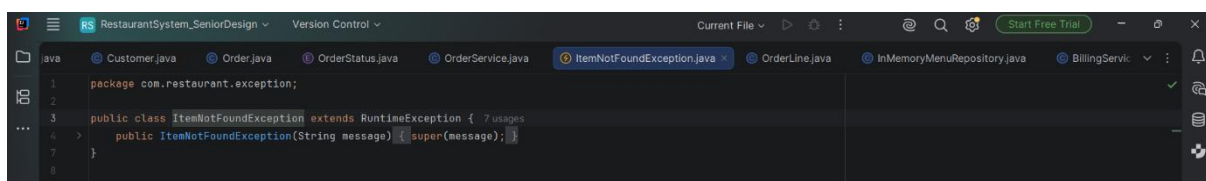


```
1 import com.restaurant.model.Customer;
2 import com.restaurant.model.MenuItem;
3 import com.restaurant.model.Order;
4 import com.restaurant.model.OrderLine;
5 import com.restaurant.model.OrderStatus;
6
7 import java.util.concurrent.atomic.AtomicInteger;
8
9 public class OrderService {
10     private final AtomicInteger orderCounter = new AtomicInteger(1);
11
12     public Order startOrder(Customer customer) {
13         if (customer == null) {
14             throw new InvalidOrderException("Cannot start an order without a registered customer");
15         }
16         return new Order("O" + orderCounter.getAndIncrement(), customer);
17     }
18
19     public void addItemToOrder(Order order, MenuItem item, int quantity) {
20         if (!item.isAvailable()) {
21             throw new InvalidOrderException(item.getName() + " is currently unavailable");
22         }
23         order.addLine(new OrderLine(item, quantity));
24     }
25
26     public void confirmOrder(Order order) {
27         if (order.isEmpty()) {
28             throw new InvalidOrderException("Cannot confirm an order with no items");
29         }
30         order.setStatus(OrderStatus.CONFIRMED);
31     }
32 }
```

Figure 2.12

Figure 2.13: ItemNotFoundException.java

Figure 2.13 is an example of one of the three custom exception classes in this version, which is a subclass of the Java Exception class RuntimeException to describe an error scenario where the requested menu item is not found. In Version A, there are no custom exception classes at all, nor does it contain proper error handling mechanisms.



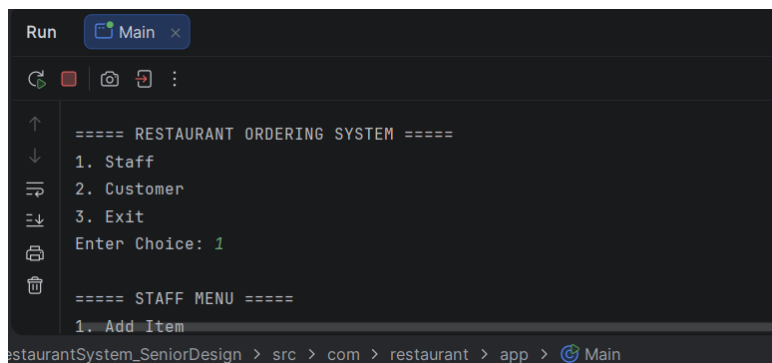
```
1 package com.restaurant.exception;
2
3 public class ItemNotFoundException extends RuntimeException {
4     public ItemNotFoundException(String message) {
5         super(message);
6     }
7 }
8
```

Figure 2.13

Outputs in Version B

Figure 2.14: Main Menu Output

Figure 2.14 shows the main menu of the AI-assisted Restaurant Ordering System, where the user selects between Staff and Customer options to access the corresponding functionality. This confirms that the RestaurantApp class correctly runs its menu loop and accepts user input, matching the intended program flow.



```
Run Main x
===== RESTAURANT ORDERING SYSTEM =====
1. Staff
2. Customer
3. Exit
Enter Choice: 1
===== STAFF MENU =====
1. Add Item
restaurantSystem_SeniorDesign > src > com > restaurant > app > Main
```

Figure 2.15: Order Confirmation Output

In Figure 2.15 is illustrated the output when a customer places an order that contains several items. It can be seen that the OrderService was successful in validating the customer and the menu items and adds the respective items as individual orders. Hence, it can be concluded that version B allows for ordering several menu items at once.

```

Run Main x
=====
Enter Item Name: Cheese Pizza
Enter Quantity: 4
Added: Cheese Pizza x4
Order another item? (y/n): y

===== MENU =====
P101 | Cheese Pizza | Pizza | Rs. 1800.00 | Available
B101 | Chicken Burger | Burger | Rs. 1200.00 | Available
D101 | Coca Cola | Drink | Rs. 350.00 | Available
=====
Enter Item Name: Coca Cola
Enter Quantity: 2
Added: Coca Cola x2
Order another item? (y/n): n

Order confirmed!

```

Figure 2.15

Figure 2.16: Receipt/Invoice Printout

Figure 2.16 depicts the detailed invoice printout generated by the ReceiptPrinter that includes each individual item along with its total price prior to showing the totals such as sub-total, taxes, discount, and grand total that have been computed by the Bill object. In contrast to Version A, which adds the discount to calculate the grand total, the grand total in this case is correctly subtracted from the sub-total and taxes.

```

Order confirmed!

===== BILL =====
Bill ID : BL101
Customer : Ruchitha Vithana (C101)
-----
Cheese Pizza      x4   = Rs. 7200.00
Coca Cola         x2   = Rs. 700.00
-----
Subtotal         : Rs. 7900.00
Tax               : Rs. 790.00
Discount         : Rs. 395.00
Grand Total      : Rs. 8295.00
=====

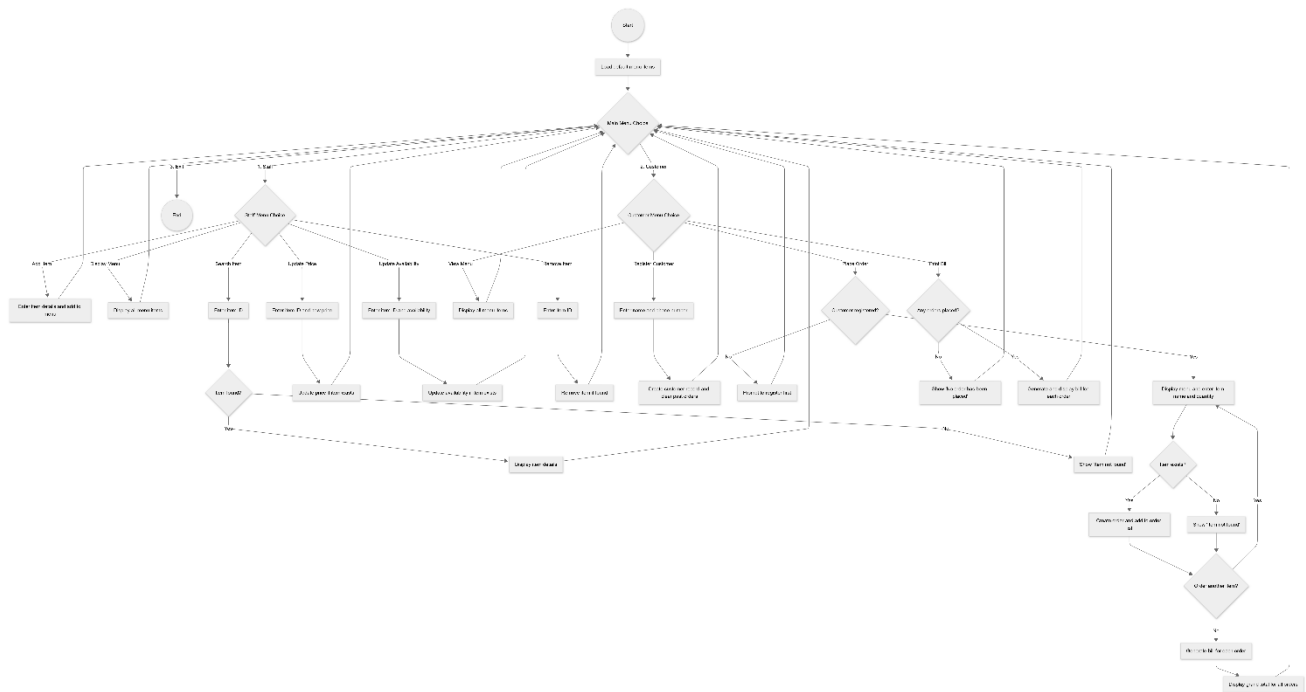
```

Figure 2.16

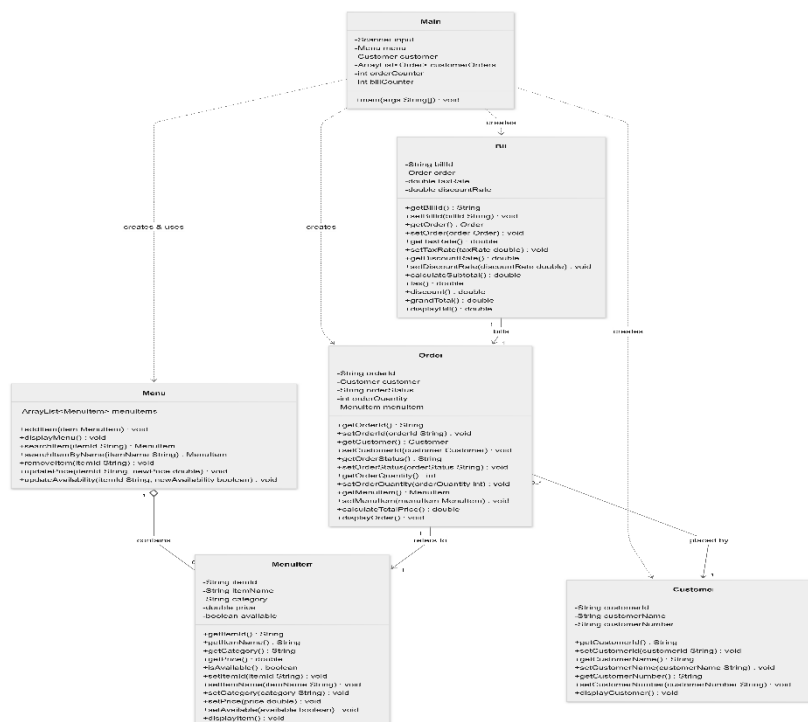
Comparison between Version A and Version B

Aspect	Version A	Version B
Coding Time	3 days	4 hours
Architecture	Single flat package, all classes together	Refined through further prompt into a layered structure (model, service, repository, exception packages)
Code Quality	Functional but simpler; all logic contained in a single flat package with no separation between business logic and presentation	Higher overall quality after refinement — organized into layered packages (model, service, repository, exception, ui) following separation of concerns
Readability & Maintainability	Reasonably readable for a small project, but harder to extend since all classes and responsibilities are mixed together	More maintainable due to clear package boundaries and interface-based design (e.g. MenuRepository), making it easier to modify or extend individual layers without affecting others
Error Handling	Minimal; relies on default Java behavior, and invalid input (e.g. non-numeric entries) can crash the program	Custom exception classes (e.g. ItemNotFoundException, InvalidOrderException) and input validation loops that recover from invalid input instead of crashing

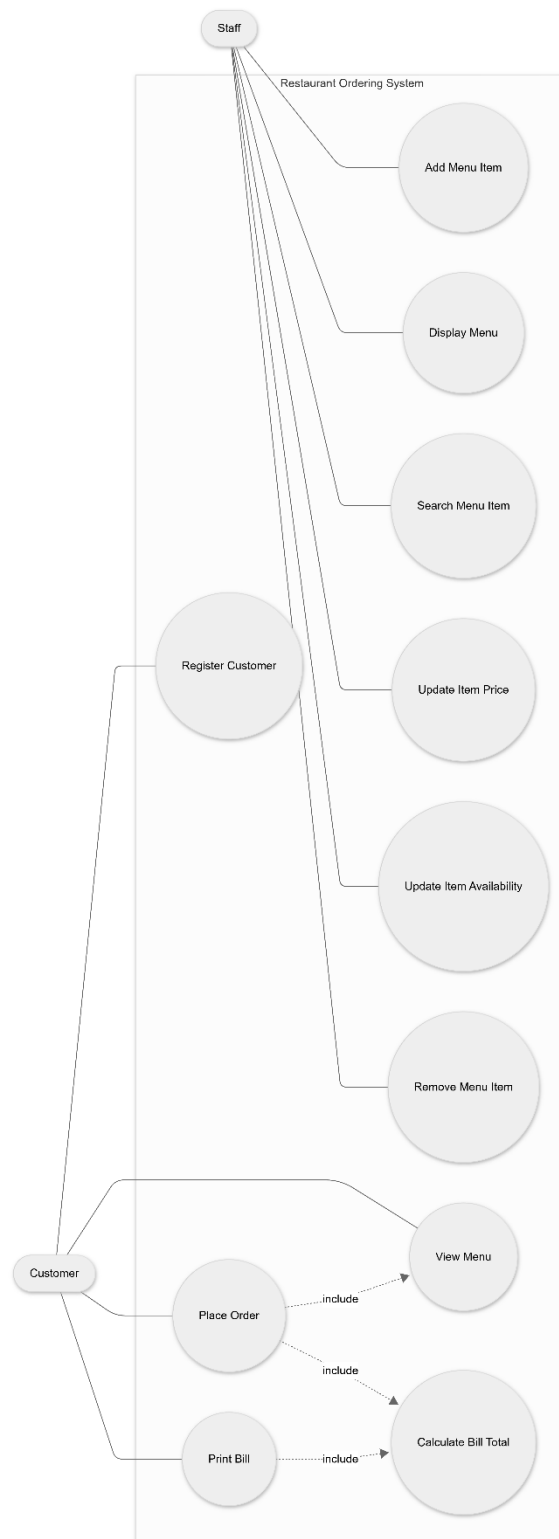
Activity Diagram of Restaurant Ordering System



Class Diagram



Use Case Diagram



Task 3 – Critical Analysis

3.1 How AI changed my coding approach?

Using AI in the development process allowed switching from coding to specifying the desired behavior and reviewing what would be generated. It was no longer necessary to design classes (like Order or Bill) step-by-step, as it was previously done in the Version A. With Version B, the coding process turned into a dialogue, where a requirement ("a customer should be able to place several orders") would be stated, the AI would try to code it, the code would be run and corrected if something did not work properly.

It affected not only how design errors were found, but in what sequence they would appear. While in Version A, the mistakes in design often appeared during coding itself, as it required making the decisions on how to structure the code, in Version B, the design mistakes only appeared during the test and correction phase of the coding cycle the mistake like an incorrect discount calculation made in Bill class would not have been detected while simply reading the code, but only after testing the application and verifying the printed result.

3.2 Whether AI Improved or Reduced Understanding

The use of AI had a positive and negative impact on understanding. On the one hand, the AI assistance helped to understand the software design patterns which could not be understood independently and would take some time. The project was reorganized into several packages – model, service, repository, exception, Ui, and the explanation of why there are such layers gave more insight about separation of concerns and interface-based development than just studying about it.

On the other hand, there was always a chance that I would accept the generated code without completely understanding it, especially when talking about small fixes like exception handling and order list built on Array List structure. In other words, it was exactly the kind of dependency danger described in Task 1. It is possible to copy the working code without even knowing what the code does. To prevent that, all changes were checked and when something was not clear enough, the explanation was asked before implementing it into the project.

3.3 Cases Where AI Should Not Be Used

As far as this particular project goes, however, AI help was particularly helpful with structural and repetitive tasks such as generating repetitive menu-driven switch statements, reorganizing the project from a flat structure into packages, and pinpointing a certain bug after its description. It was not as dependable, moreover, in such cases as:

Checking the correctness of business logic implementation, such as determining that some tax and discount calculations are implemented in a right way. An erroneous formula may look plausible enough, just like it happened in case of the discount calculations.

Trying to learn a new concept without cross-verification of information provided by AI, because an erroneous explanation is as fluent as a right one.

Any assignment or exercise meant to test a student's ability to do something on their own, such as Version A of this particular project, when any use of AI would contradict the main idea of the comparison.

Code that is either safety-critical, security-critical, or finance-critical, when an unverified logical mistake will have effects outside the scope of a class project.

3.4 Ethical Considerations in AI-Generated Code

Several ethical concerns can be raised regarding this particular project. First, there is the question of transparency this report clearly shows which parts of Version B have been developed with the help of the AI tool as opposed to the independent development of AI-produced code. The second concern relates to accountability: while the AI developed the code, it was the developer's duty to ensure its correctness, including compiling it, testing the ordering flow, and debugging the discount function. Submitting any AI-produced code without prior verification would amount to neglecting one's accountability duties regardless of the method used to develop the code. Thirdly, there is the matter of academic integrity when it comes to an educational assignment, as it is quite obvious that AI code should not be submitted in an assessment aimed at testing one's programming skills without the use of any assistance. Finally, the matter of code ownership and licensing should be addressed: coding assistants are trained

on large amounts of previously written code, and developers using such code in their real-world applications should understand that originality and licensing remain open questions in the industry, even though they did not materially affect a self-contained academic project like this one.

Task 4 – Conclusion

The current task highlighted the advantages and disadvantages of AI-assisted coding using the direct comparison of two versions of the same Restaurant Ordering System. Version A, which was not assisted by AI, was done much slower compared to version B, but helped the student strengthen his knowledge of basic coding techniques such as manual class relations design and logical error debugging. Version B, which was AI-assisted, was done much faster and resulted in a better architecture of the system, as it split it into model, service, repository, exception, and UI layers. However, this resulted in the need for checking, testing, and, in some cases, correcting the code, because the discount calculation was incorrect at first.

The project confirms the thesis that AI-based programming should be seen as a productivity aid rather than a substitution for skills in programming. The use of AI increased the speed of routine and structural operations, as well as provided better design patterns, resulting in higher quality of the whole system. On the other hand, it did not eliminate the necessity of knowledge on the part of the developer concerning what is being coded: tests, validation of the results, and critical thinking about any unusual operation were necessary all along.

Repository Link

GitHub Repository Link = <https://github.com/RuchithaAV/Restaurant-Ordering-System.git>